

Rahasya build plan

Sep 27, 2026 · @Kumar

Summary

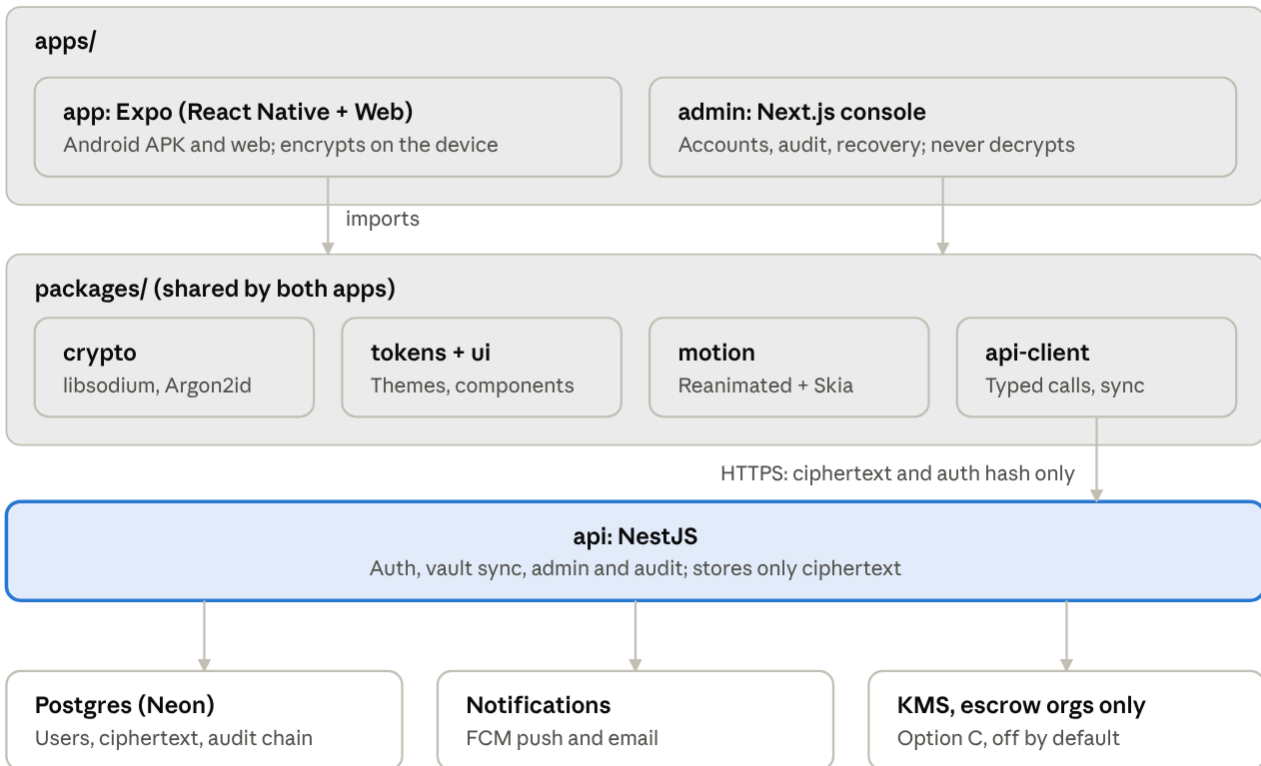
Rahasya v1 ships in about 28 weeks of full-time solo work as one Expo codebase (Android APK and web), a NestJS API and a separate admin console, built encryption-first. The zero-knowledge crypto core is built and tested before any screen that uses it, so no plaintext ever reaches the server by accident.

- **Scope:** all P0 requirements from the PRD plus the P1 items the designs rely on (FR-01 to FR-38, apart from breach checks, import and export, and autofill), the 11 phone screens, the desktop web layout, the admin console and the 13 approved animations. Phase 2 items (biometrics beyond unlock, breach checks, import and export) stay out.
- **Order of work:** repository and tokens, then crypto core and auth, then vault sync, then screens, then motion, then admin and emergency access, then security review and beta.
- **Who and cost:** one developer (you), at zero cost: free tiers only, no paid reviewers or services. Part-time work stretches the timeline (about double at 20 hours a week), but the order stays the same.

Architecture and monorepo layout

One pnpm and Turborepo monorepo holds both apps, the API and four shared packages, so the phone app, web app and admin console use the same tokens, crypto and API client.

Plaintext stays on devices; the API stores only ciphertext



architecture · 2 apps, 4 shared packages, 1 API, 3 services

Encryption and decryption happen only inside `packages/crypto` on the user's device; the API and database never see a master password, a vault key or plaintext. The KMS link exists only for organisations that choose escrow (PRD Option C); it is off by default because cloud key services usually charge a small monthly fee.

Technology choices

Every choice favours one shared codebase and audited crypto over custom code; versions are pinned in the lockfile at repo setup.

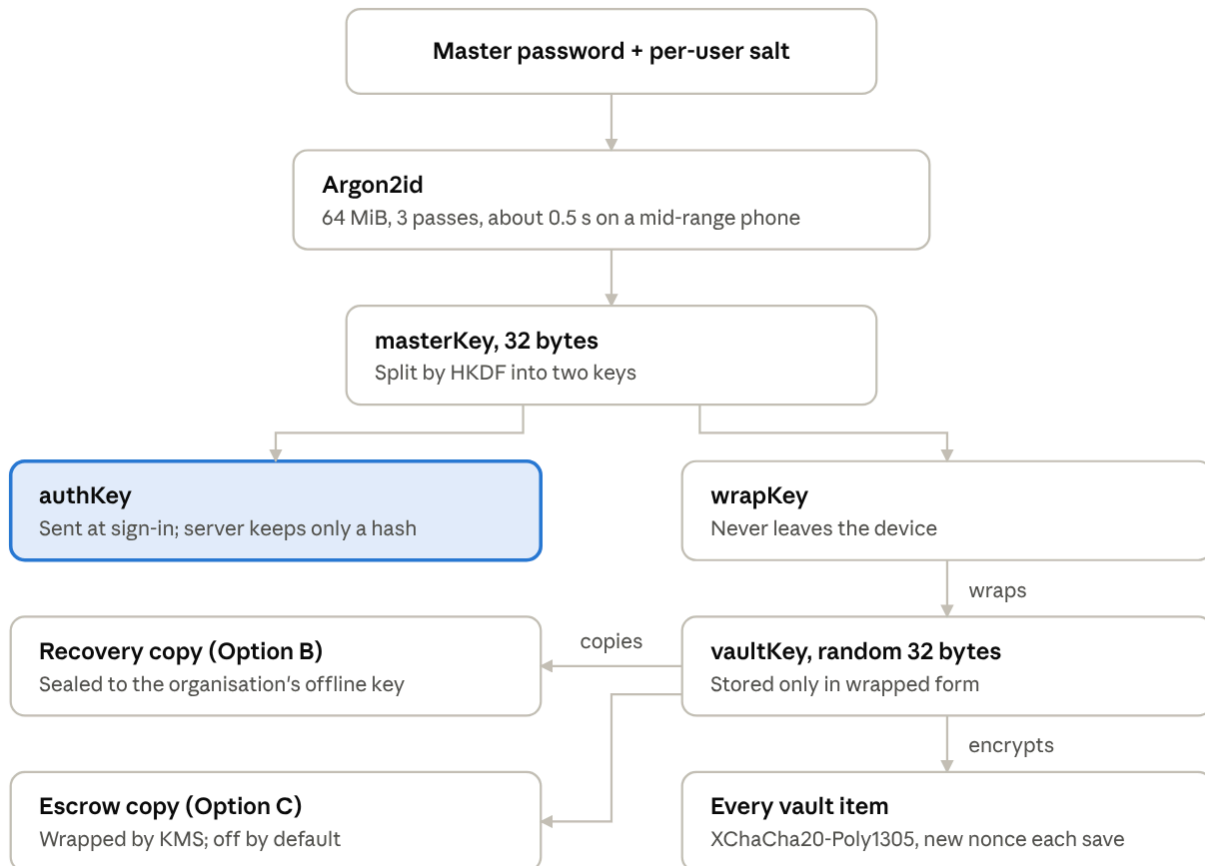
Area	Choice	Why
App	Expo (React Native + React Native Web), Expo Router	One codebase for the Android APK and web, file-based routes on both
Crypto	libsodium (react-native-libsodium on device, libsodium-wrappers-sumo on web)	Audited XChaCha20-Poly1305 and Argon2id with identical output on both platforms
Animation	Reanimated 3 and React Native Skia	120 fps on the UI thread; Skia draws the glass orb, rings and gradients

Area	Choice	Why
Secure storage	Android Keystore through expo-secure-store; IndexedDB with a non-extractable WebCrypto key on web	Keeps the wrapped key off plain storage
Screen privacy	expo-screen-capture (FLAG_SECURE), blur in the app switcher	FR-04: no screenshots or previews of secrets
Unlock	expo-local-authentication	Fingerprint unlock (M1) backed by the Keystore
API	NestJS, Prisma, class-validator	Typed modules for auth, vault, admin and audit
Database	PostgreSQL on Neon	Managed, branches for preview environments
Admin console	Next.js (App Router), strict content security policy	Server-rendered, separate origin from the vault app
Monorepo	pnpm workspaces and Turborepo	Shared packages with cached builds
Testing	Vitest, Jest, Detox (Android), Playwright (web)	Unit, crypto vectors, and end-to-end on both platforms
Hosting	Vercel Hobby (web and admin), Render free web service (API), Neon free tier (database), GitHub Actions and Releases (CI and APK downloads)	All free for a personal project; the free API sleeps when idle, so the first request after a pause is slow; move to a small VPS later if needed

Encryption core

The crypto package is built and tested first, and every other package calls it; nothing else in the codebase touches raw keys.

One master password unlocks everything; the server keeps only a hash



key hierarchy · master password to every item

Changing the master password re-wraps only the vaultKey, so items are never re-encrypted.

Ciphertext format. Every blob starts with a 1-byte version (0x01), then a 1-byte algorithm id, the 24-byte nonce, the ciphertext and the 16-byte tag. The version byte lets us change algorithms later without breaking old vaults.

Build order

1. Argon2id derivation, with parameters stored per user so they can be raised later.
2. HKDF split into authKey and wrapKey.
3. vaultKey generation and wrapping.
4. Item encrypt and decrypt with the versioned header.
5. Recovery sealing (X25519 sealed box) for users who opt in.
6. Key rotation: change master password and re-wrap.

Tests that gate every merge

- Fixed test vectors that must give byte-identical output on Android and web.
- Tamper tests: a changed byte anywhere must fail decryption, never return garbage.

- Memory hygiene: keys are zeroed after use and never logged, even in debug builds.
- A self-review against the OWASP MASVS and ASVS checklists before beta; the package uses only audited libsodium, never custom crypto, and can be open-sourced later so others can review it.

Design system in code

The three themes become one tokens package, generated from the design document's token table, so a colour changes in one place for the phone, web and admin at once.

- **packages/tokens** : one JSON file per theme (D9, D10, D14) with the shared token names (`bg`, `surface`, `text`, `text-muted`, `accent`, `accent-solid`, `on-accent`, `soft`, `warning`, `danger`, `success`, plus `orbGlow`, `spec` and `orbShadow`). A build step outputs TypeScript for the apps and CSS variables for the admin console.
- **ThemeProvider** : reads Light, Dark or Auto from settings (FR-24); Auto maps day to D10 and night to D9 or D14 by the user's choice. On web, an inline script sets the theme before first paint so there is no flash.
- **Fonts**: Outfit (D9, D14), Sora (D10) and DM Mono for secrets, bundled with the app rather than loaded from the network.

Shared components (in `packages/ui`, each with the three theme variants and a reduced-motion variant):

Component	Used on
GlassSurface (tile, card, sheet, pill)	Every screen
GlassOrb with Keyhole	Splash, unlock, lock, add, update
SecretField (mask, reveal, copy, auto-hide)	Entry detail, edit, desktop detail
FieldRow (edited, new, will-be-removed states, Undo)	Edit screen S9
AddFieldSheet (8 types, suggestions)	S10
EntryRow and WalletCard	Home, list, desktop list
GroupDrawer and GroupTree	Drawer S3, desktop sidebar
StrengthRing and StrengthBar	Generator, health, edit
Toast and ClipboardPill	Copy, delete, save
FloatingNavBar	Phone home, health, settings

Each component gets a Storybook entry in all three themes, so design review happens in code before any screen is assembled.

Motion in code

All 13 approved animations are assembled from six shared primitives in `packages/motion`, so they share timing, easing and reduced-motion behaviour by construction. The [motion library](#) is the visual reference for review.

Primitive	What it does	Built with
OrbitRings	Three rings that draw, turn, close into a circle, flatten, or strain and spring back	Skia paths, Reanimated shared values
CipherText	Scrambles and decrypts text letter by letter, or re-encrypts back to dots	Reanimated frame callback, DM Mono
GlassOrb	The orb with glow, highlight and depth; morphs into a card	Skia (shadows, gradients, blur)
GradientUnderline	Draws, counts down or turns red	Reanimated width and colour
Iris	Opens or closes a screen in a circle	Skia clip, shared transition
SharedCard	Lifts a card and grows it into a page; icon and name glide	Reanimated layout transitions

Animation	Built from
SP14 splash	OrbitRings (flatten), CipherText, GlassOrb
M1 unlock, M7 auto-lock	OrbitRings (close), GlassOrb, Iris
M2o wrong password	OrbitRings (strain and spring back), GradientUnderline
M3 copy, M4 reveal	CipherText, GradientUnderline
M5a open entry	SharedCard
A11 add, U4 update	CipherText, OrbitRings, GlassOrb (morph), SharedCard
M6 delete, M9 regenerate	CipherText, GradientUnderline
M8 theme switch	Iris

Animation	Built from
M10 pull to sync	OrbitRings (spin), GlassOrb

Rules every animation follows

- Runs on the UI thread and holds 60 fps on a mid-range Android phone (120 fps where the screen supports it).
- Tap to skip on anything over 1 second; the quicker version plays after the first three times.
- Reduced motion: final frame with a 200 ms fade.
- Paired haptics: a light tick on success, a double tap on failure (M2o).

Backend API and database

The API is a sync server for ciphertext plus account and admin management; no endpoint accepts or returns plaintext secrets.

Area	Endpoints	Notes
Sign-up and sign-in	POST /auth/register , POST /auth/prelogin , POST /auth/login , POST /auth/refresh , POST /auth/logout	Prelogin returns the user's salt and Argon2id settings; login checks the authKey hash; short JWT plus rotating refresh token
Two-step sign-in	POST /auth/2fa/setup , POST /auth/2fa/verify	TOTP; rate-limited
Vault	GET /vault/sync?since= , PUT /vault/items/:id , DELETE /vault/items/:id	Encrypted blobs with a revision number; conflicts resolved by newest revision, older version kept in history
Groups and labels	GET/PUT /vault/groups , GET/PUT /vault/labels	Names are encrypted too
Devices	GET /devices , DELETE /devices/:id	Sign out a lost phone
Admin	GET /admin/users , POST /admin/users/:id/lock , POST /admin/users/:id/suspend , POST /admin/invites	Admin role only; separate origin

Area	Endpoints	Notes
Emergency access	POST /admin/recovery, POST /admin/recovery/:id/confirm, POST /recovery/:id/cancel	Security-key confirmation, 48-hour wait, user can cancel
Audit	GET /admin/audit	Read-only; hash-chained entries
Admins and roles	GET/POST /admin/admins, PATCH/DELETE /admin/admins/:id, GET/POST/PATCH /admin/roles	Super admin and Owners only; any change to the super admin returns 403, except a hand-over by the super admin; every change logged

Table	Holds
users	Email, salt, Argon2id settings, authKey hash, wrapped vaultKey, 2FA secret (encrypted), role
vault_items	User, item id, encrypted blob, revision, deleted flag, timestamps
item_history	Previous encrypted versions for password history and undo (30 days)
groups, labels, item_labels	Encrypted names and links
devices, refresh_tokens	Signed-in devices and token rotation
recovery_keys	Vault keys sealed for recovery (Option B) or escrow (Option C)
recovery_requests	Requester, reason, confirmed at, waiting-period end, status
audit_log	Actor, action, target, device, result, previous entry's hash
admins, admin_roles	Admin accounts, their security keys and role; roles with their permission lists (built-in and custom)

Rate limits: 5 failed unlocks per account then a 30-second wait (matching M2o), and IP-based limits on sign-in and prelogin.

Admin console and emergency access

The admin console is its own Next.js app on its own domain, with no vault code in its bundle except the recovery step, which loads only during a confirmed recovery.

- **Sign-in:** admin account with master password plus a hardware security key (WebAuthn, required); sessions expire after 15 minutes; sign-in only from allow-listed IP addresses; the super admin is created with a server command, then the super admin and Owners add admins in the console.
- **Roles:** Super admin (one per organisation, permanent, everything), Owner (everything, including admins and roles), Admin (accounts and recovery) and Auditor (read-only audit log), plus custom roles that the super admin or Owners create.
- **Hardening:** strict content security policy, no third-party scripts, every action written to the audit log before it runs.

Emergency access flow (PRD Option B, and Option C for escrow organisations)

Step	Who	Safeguard
1. Request recovery with a written reason	An admin whose role has the recovery permission	Logged before it is saved
2. Confirm with a security key or passkey	The same admin, on web or phone	Proves it is really them; starts the wait
3. 48-hour waiting period	System	User notified by email and on every device; user can cancel
4. Recover with the offline key	An admin with the recovery permission, on their own computer	The offline key is loaded locally and never uploaded; the server cannot decrypt
5. User sets a new master password	User	Old sessions signed out; recovery copy re-sealed

One admin is enough to run a recovery. The protection comes from the security-key confirmation, the waiting period the user can cancel, the notifications, the offline key and the tamper-evident audit log. If the user cancels during the wait, the request closes and stays in the audit log. For Option C, step 4 unwraps with the KMS key instead, and the organisation is told this at setup.

Admins and roles

- The super admin is created with a server command at setup; after that, only the super admin and Owners add admins, change roles or remove admins, in the console.

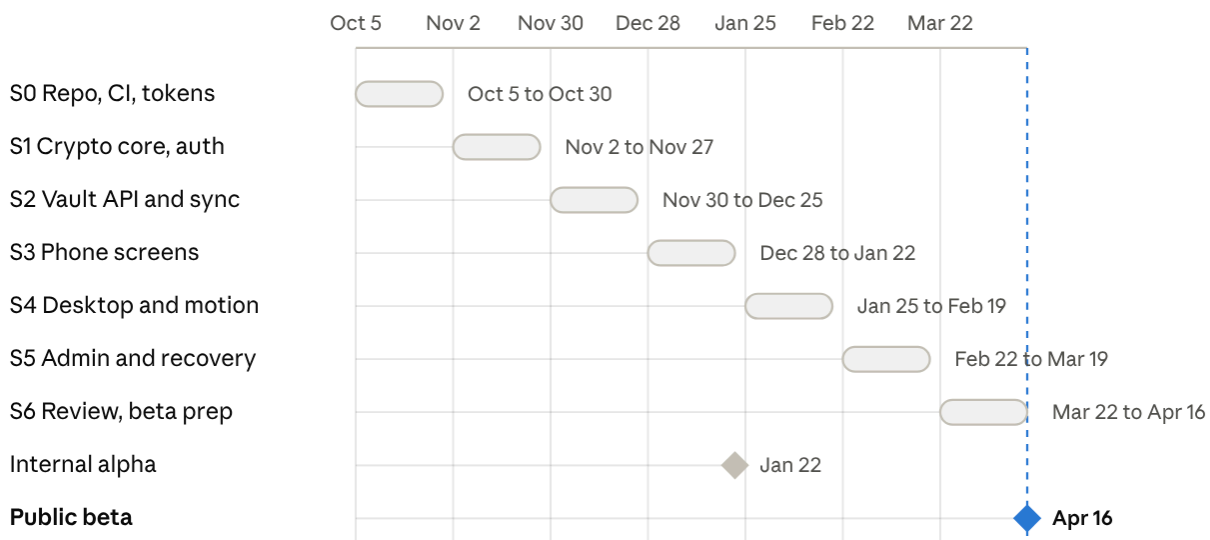
- New admins are invited by email and must register a hardware security key at first sign-in.
- Built-in roles: Owner (everything, including admins and roles), Admin (accounts and recovery) and Auditor (read-only audit log). Owners can also create custom roles by choosing permissions: manage users, reset two-step sign-in, start recovery, urgent decrypt, view audit log, manage admins.
- The super admin can never be deleted, demoted, suspended or locked from the console. It is stored as a protected flag on the admin account, and the API refuses any change to it with 403. Only the super admin can hand the role to another admin, confirmed with their own security key. A break-glass server command (run on the server with the offline key) can reset a lost key or freeze a compromised super admin, but never delete it. Every change is logged and all Owners are notified.

Milestones and sprints

Seven four-week sprints, sized for one developer working full-time, take Rahasya from an empty repository to a public beta; the internal alpha at the end of S3 is the first build people use daily.

Public beta lands about 28 weeks after the repository is set up

Four-week sprints for one developer working full-time, from 5 Oct 2026



build timeline · 7 sprints, 2 milestones

Sprint	Done when
S0	CI green on every push; tokens build for app and admin; Storybook shows GlassSurface in 3 themes

Sprint	Done when
S1	Register, sign in, unlock and lock work end to end; crypto vectors pass on Android and web
S2	Items sync between two devices; offline edits merge; history keeps old versions
S3	All 11 phone screens work with real data (internal alpha)
S4	Desktop layout done; all 13 animations match the motion library
S5	Admin console live on staging; a full recovery rehearsed end to end
S6	Security self-review closed with no high or critical findings; APK on GitHub Releases and web launch ready

The dates assume a 5 October 2026 start and one developer working full-time; Diwali week (early November) falls in S1 and the year-end holidays in S2 and S3, so plan a few days of slack there. At about 20 hours a week, expect roughly double.

Testing and security review

Nothing ships to beta until the crypto package and the recovery flow pass a security self-review with free tools and checklists, and every finding rated high or critical is fixed.

Layer	What is tested	Tool	Gate
Crypto	Test vectors, tamper, wrong key, key rotation	Vitest on web, Jest on device	Every merge
API	Auth, sync conflicts, rate limits, admin roles	Jest with a Neon test branch	Every merge
Screens	Each component in 3 themes, reduced motion, large text	Storybook, visual snapshots	Every merge
End to end	Sign up, add, edit, copy, sync on 2 devices, recovery	Detox (Android), Playwright (web)	Nightly and before release
Performance	60 fps for all animations, unlock under 1 s after Argon2id	Android profiler on a mid-range phone	Each sprint demo

Layer	What is tested	Tool	Gate
Accessibility	Screen readers, contrast in all 3 themes, 44 px touch targets	TalkBack, axe	Before beta

Security work

- Threat model written in S0 and updated each sprint (lost phone, malicious admin, compromised server, phishing).
- Static analysis and dependency scanning on every pull request with free tools (CodeQL, Semgrep, npm audit); no dependency added without review.
- No secrets, keys or plaintext in logs, crash reports or analytics; checked by an automated test.
- Self-review of `packages/crypto`, the API and emergency access in S6 against OWASP MASVS and ASVS, plus a free OWASP ZAP scan of the staging API and admin console.

CI/CD, environments and release checklist

Every pull request gets a free Vercel preview that talks to the shared staging API, and only a green main branch can reach production.

Environment	App	Admin	API and database
Preview (per pull request)	Vercel preview; Android build on request	Vercel preview	Shared staging API (free hosts have no per-PR previews)
Staging	Web on a Vercel staging URL; test APK from CI	Staging domain	Render free service, Neon staging branch
Production	Web at the main domain; signed APK on GitHub Releases (Play Store later, which has a one-time developer fee)	Admin domain, allow-listed	Render free service, Neon free tier; nightly encrypted database backup with GitHub Actions

- **Pipeline:** lint, type check, unit and crypto tests, build, then deploy to preview; main deploys to staging; a tagged release promotes to production.
- **Android:** EAS Build's free tier, or a local Gradle build, signs the APK; the signing key lives in EAS or your password manager, plus a sealed offline backup.
- **Database:** Prisma migrations run before the new API version starts; every migration must be backward compatible for one release.

Release checklist

- ☐ All tests green, including nightly end-to-end runs
- ☐ Crypto vectors identical on Android and web
- ☐ No high or critical security findings open
- ☐ Animations checked against the motion library on a mid-range phone
- ☐ Database backup and restore tested in the last 7 days
- ☐ Release notes and GitHub Release updated
- ☐ Rollback plan written: previous API version and app build ready

Risks and mitigations

The biggest risk is a crypto mismatch between Android and web, so it is tested first and on every merge.

Risk	Impact	Mitigation
Crypto output differs between Android and web	A vault saved on one device cannot open on the other	Shared test vectors from S1; the same libsodium build on both
Argon2id too slow on low-end phones	Unlock feels broken	Tune per device class in S1; parameters stored per user so they can change
Skia animations drop frames on older phones	Premium feel lost	Profile each primitive in S4; lighter fallback (no blur) below a device threshold
Sync conflicts lose an edit	User data loss	Newest revision wins, older kept in history; tested with two devices offline
Admin misuses emergency access	Trust lost	Security-key confirmation, 48-hour cancellable wait, user notified on every device, offline key only, recovery permission limited to chosen roles, audit log
User forgets master password with no recovery set up	Vault permanently lost	Clear warning at sign-up; encourage opt-in recovery and a printed recovery kit

Risk	Impact	Mitigation
A security flaw is missed without an outside reviewer	Beta slips	Audited libsodium only; self-review against OWASP checklists from S1; free scanners on every merge; optionally open-source the crypto package for community review
Super admin's key lost or stolen	The one account that can never be removed is out of reach, or in the wrong hands	Register two security keys at setup; break-glass server command can reset the key or freeze the account without deleting it; hand-over to another admin; alerts to all Owners
Free hosting sleeps or changes its limits	The first sync after a pause can take up to about a minute; a free plan could end	Show a "waking up the server" state; keep the API portable as a Docker image so it can move hosts; check each provider's free terms before launch

First week tasks

By the end of week one, a pull request should build, test and deploy a preview of an app that shows a glass tile in all three themes.

- ☐ Create the monorepo (pnpm, Turborepo) with `apps/app`, `apps/admin`, `apps/api` and the four packages
- ☐ Set up CI: lint, type check, tests and preview deploys on every pull request
- ☐ Create free Vercel, Render and Neon projects for staging and production
- ☐ Write `packages/tokens` from the design document's token table and generate TypeScript and CSS
- ☐ Build ThemeProvider with Light, Dark and Auto, and the no-flash script for web
- ☐ Add Storybook with GlassSurface and GlassOrb in all three themes
- ☐ Spike libsodium on Android and web: Argon2id and XChaCha20-Poly1305 give identical output
- ☐ Write the first threat model and turn on free scanning (CodeQL, Semgrep, npm audit)
- ☐ Confirm the start date and your weekly hours so the sprint dates can be fixed